

jueves 12 febrero 2026

Hola José! Esta semana hice lo acordado con respecto al dataset online, incluyendo el aumento de datos en caché; continúe “refactorizando” y organizando el código para los experimentos, tanto que ya ni me hace falta el `git` o el `tmux` y adicionalmente hice pruebas con una nueva arquitectura. Sobre el dataset, no tuve mucho problema y realmente va bastante más rápido. Valió la pena la charla. Sobre lo de refactorizar, ahora estoy muy cómodo y me supone poco esfuerzo hacer cualquier prueba. Igual es un poco innecesario pero como no se que pruebas vamos a querer y aun soy muy joven e ilusionado, pues simplemente me apetece dedicar mi tiempo en eso :) Antes de profundizar en la nueva arquitectura te quiero enseñar la sección del artículo donde hablan de la visualización.

We recapitulate the method to visualize attention maps in ??, at first specializing their use to instances of the CRATE model before generalizing to the ViT. For the k^{th} head at the ℓ^{th} layer of CRATE, we compute the *self-attention matrix* $\mathbf{A}_k^\ell \in \mathbb{R}^n$ defined as follows:

$$\mathbf{A}_k^\ell = \begin{bmatrix} A_{k,1}^\ell \\ \vdots \\ A_{k,N}^\ell \end{bmatrix} \in \mathbb{R}^N, \quad \text{where} \quad A_{k,i}^\ell = \frac{\exp(\langle \mathbf{U}_k^{\ell*} \mathbf{z}_i^\ell, \mathbf{U}_k^{\ell*} \mathbf{z}_{\text{cls}}^\ell \rangle)}{\sum_{j=1}^N \exp(\langle \mathbf{U}_k^{\ell*} \mathbf{z}_j^\ell, \mathbf{U}_k^{\ell*} \mathbf{z}_{\text{cls}}^\ell \rangle)}. \quad (1)$$

We then reshape the attention matrix $\mathbf{A}_k^\ell$ into a $\sqrt{n} \times \sqrt{n}$ matrix and visualize the heatmap as shown in (Figura 20, que es lo que queremos tu y yo). For example, the i^{th} row and the j^{th} column element of the heatmap in (Figura 20, que es lo que queremos tu y yo) corresponds to the m^{th} component of $\mathbf{A}_k^\ell$ if $m = (i - 1) \cdot \sqrt{n} + j$. In (Figura 20, que es lo que queremos tu y yo), we select 4 attention heads of CRATE and visualize the attention matrix $\mathbf{A}_k^\ell$ for each image.

O sea, no parece que esté invirtiendo la red. Leyendo eso tampoco tengo claro porque habría resolución más allá del token. Estoy confuso con esto. De todas formas aún no implementé ninguna versión. Lo dejo para la próxima se-

mana, o igual podríamos volver a hablarlo.

1 Arquitectura propuesta

¿Por qué?

En el artículo aproxima una matriz de forma $(I - A)$ como $(I + A)$, lo cual corresponde con el primer orden de https://en.wikipedia.org/wiki/Neumann_series, lo cierto es que no conocía esto previamente y por tanto lo mire y lo entendí. De forma natural me pregunté, ¿Por qué no usar más orden? Intenté responder con lo que diría Yi Ma:

- Primeramente pensé que era mala idea porque sería mas caro. Pero realmente no lo es mucho más y si bien vale la pena pues el coste computacional no es motivo.
- Después pensé en la simpleza. Yi Ma parece defender que CRATE es una obra científica y no de ingeniería, por tanto parecería natural quedarse sólo con lo más fundamental y obviar los bells and whistles como dirías tú. Pero realmente lo más científico es aproximar bien.
- Finalmente (ahora) pienso que Yi Ma, llegó al Transformer no de casualidad si no que lo deseaba, quería justificar lo conocido. Este me parece el motivo por el que Yi Ma se quedó en el primer orden de Neumann.

Ahora te voy a explicar exactamente de que estoy hablando y que arquitectura nueva se deriva.

Primero te muestro lo original por referencia. Aquí esta la matriz a invertir

$$\nabla R^c(\mathbf{Z} \mid \mathbf{U}_{[K]}) = \beta \sum_{k=1}^K \mathbf{U}_k (\mathbf{U}_k^* \mathbf{Z}) \left(\mathbf{I} + \beta (\mathbf{U}_k^* \mathbf{Z})^* (\mathbf{U}_k^* \mathbf{Z}) \right)^{-1}. \quad (2)$$

Aquí la aproximación

$$\nabla R^c(\mathbf{Z} \mid \mathbf{U}_{[K]}) \approx \beta \sum_{k=1}^K \mathbf{U}_k (\mathbf{U}_k^* \mathbf{Z}) (\mathbf{I} - \beta (\mathbf{U}_k^* \mathbf{Z})^* (\mathbf{U}_k^* \mathbf{Z})) \quad (3)$$

$$= \beta \left(\sum_{k=1}^K \mathbf{U}_k \mathbf{U}_k^* \right) \mathbf{Z} - \beta^2 \sum_{k=1}^K \mathbf{U}_k (\mathbf{U}_k^* \mathbf{Z}) (\mathbf{U}_k^* \mathbf{Z})^* (\mathbf{U}_k^* \mathbf{Z}). \quad (4)$$

Finalmente

$$\mathbf{Z}^{\ell+1/2} = (1 - \beta\kappa)\mathbf{Z}^\ell + \beta\kappa\text{MSSA}\mathbf{Z}^\ell \mid \mathbf{U}_{[K]}^\ell \approx \mathbf{Z}^\ell - \kappa\nabla R^c(\mathbf{Z}^\ell \mid \mathbf{U}_{[K]}^\ell), \quad (5)$$

Puedes revisar las definiciones de SSA y MSSA aquí <https://arxiv.org/pdf/2311.13110>

Ahora con el segundo orden la inversa quedaría (si entras en la wikipedia, T sería $-\beta(\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z})$):

$$\nabla R^c(\mathbf{Z} \mid \mathbf{U}_{[K]}) \approx \beta \sum_{k=1}^K \mathbf{U}_k(\mathbf{U}_k^*\mathbf{Z})(\mathbf{I} - \beta(\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z}) + \beta^2((\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z}))^2) \quad (6)$$

$$= \beta \left(\sum_{k=1}^K \mathbf{U}_k \mathbf{U}_k^* \right) \mathbf{Z} - \beta^2 \sum_{k=1}^K \mathbf{U}_k(\mathbf{U}_k^*\mathbf{Z})(\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z}) + \beta^3 \sum_{k=1}^K \mathbf{U}_k(\mathbf{U}_k^*\mathbf{Z})((\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z}))^2. \quad (7)$$

$$= \beta \mathbf{Z} - \beta^2 \sum_{k=1}^K \mathbf{U}_k(\mathbf{U}_k^*\mathbf{Z})(\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z}) + \beta^3 \sum_{k=1}^K \mathbf{U}_k(\mathbf{U}_k^*\mathbf{Z})((\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z}))^*((\mathbf{U}_k^*\mathbf{Z})^*(\mathbf{U}_k^*\mathbf{Z})). \quad (8)$$

Tomándonos la licencia de que $A^2 = A^t A$ es decir asumimos simetria en $U^t Z$. Que el sumatorio de utu es I ya lo dice el, pero lo pongo explitamente por espacio.

El tema esque ahora tenemos una atencion más. con signo opuesto

2 experimentos

bueno pues lo implemente

hice experimentos con configuraciones identicas de mi red y la original.

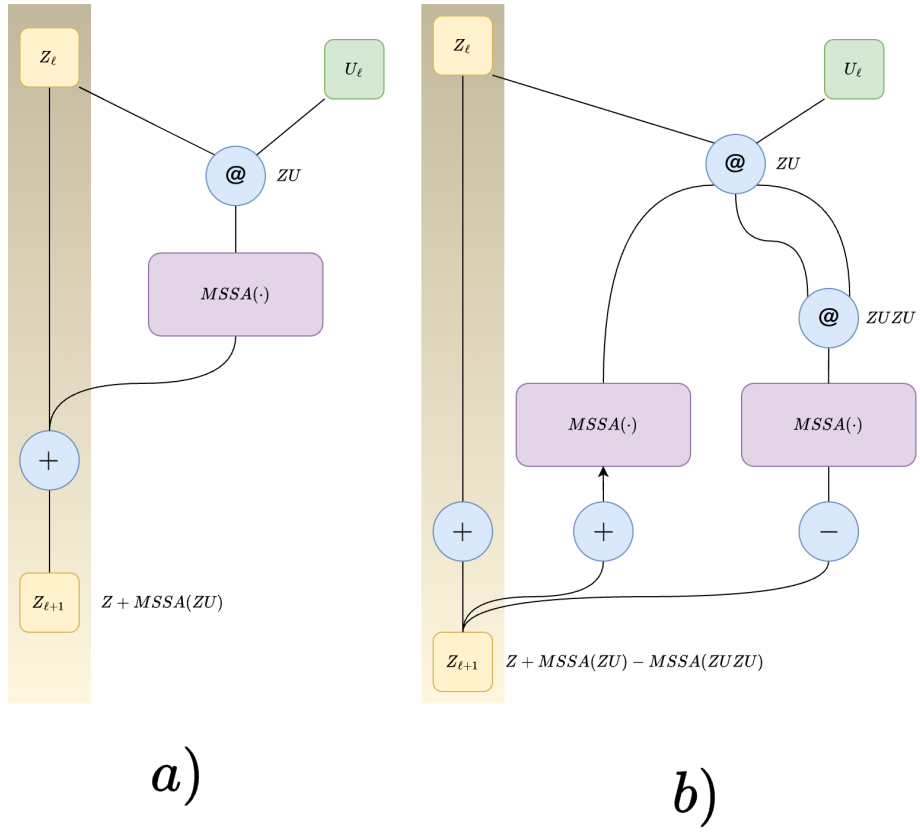
```

1 class Attention(nn.Module):
2     def __init__(self, dim, heads=8, dim_head=64, dropout=0., order
      = 'first'):
3         super().__init__()
4         inner_dim = dim_head * heads
5         project_out = not (heads == 1 and dim_head == dim)
6
7         self.heads = heads
8         self.scale = dim_head ** -0.5
9         self.order = order # 'first' or 'second'
10
11         self.attend = nn.Softmax(dim=-1)
12         self.dropout = nn.Dropout(dropout)
13
14         self.qkv = nn.Linear(dim, inner_dim, bias=False)
15
16         self.to_out = nn.Sequential(
17             nn.Linear(inner_dim, dim),

```

Figure 1: a) es lo original b) lo que propongo

$$U \doteq K = Q = V$$



```

18         nn.Dropout(dropout)
19     ) if project_out else nn.Identity()
20
21     def forward(self, x):
22         w = rearrange(self.qkv(x), 'b n (h d) -> b h n d', h=self.
heads)
23
24         # Compute (U^T Z)^T (U^T Z)
25         dots = torch.matmul(w, w.transpose(-1, -2)) * self.scale
26
27         if self.order == 'first':
28             # First-order Neumann approximation
29             # out = (U^T Z) * softmax((U^T Z)^T (U^T Z))
30             attn = self.attend(dots)
31             attn = self.dropout(attn)
32             out = torch.matmul(attn, w)
33
34         elif self.order == 'second':
35             # Second-order Neumann approximation
36             # out = out_1st - out_2nd
37
38             # First order term: (U^T Z) * softmax((U^T Z)^T (U^T Z)
39             attn_1st = self.attend(dots)
40             attn_1st = self.dropout(attn_1st)
41             out_1st = torch.matmul(attn_1st, w)
42
43             # Second order term: (U^T Z) * softmax(((U^T Z)^T (U^T
Z))^2)
44             # Compute ((U^T Z)^T (U^T Z))^2
45             dots_2nd = torch.matmul(dots, dots)
46             attn_2nd = self.attend(dots_2nd)
47             attn_2nd = self.dropout(attn_2nd)
48             out_2nd = torch.matmul(attn_2nd, w)
49
50             # Combine: subtract second order correction
51             out = out_1st - out_2nd
52
53         else:
54             raise ValueError(f"order must be 'first' or 'second',
got {self.order}")
55
56         out = rearrange(out, 'b h n d -> b n (h d)')
57         return self.to_out(out)

```

Listing 1: la atencion

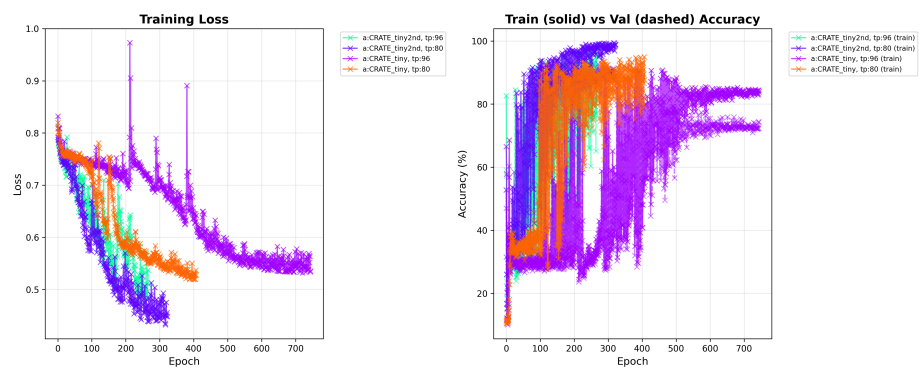


Figure 2: sparsidad de mi red

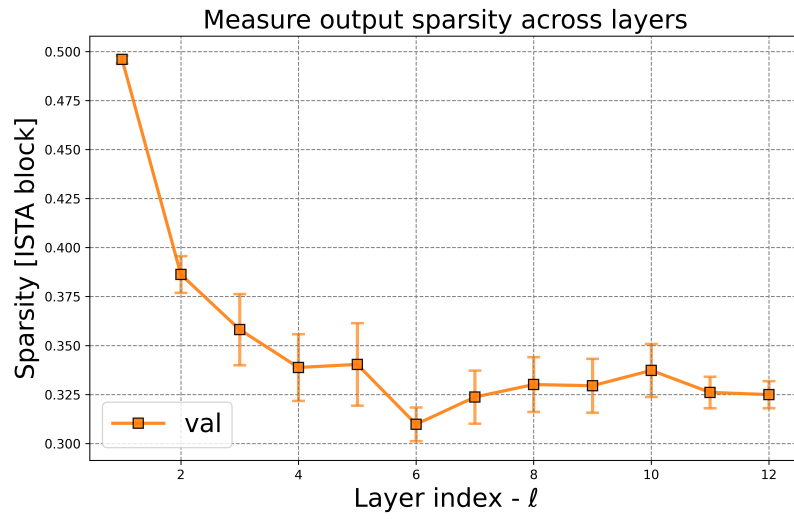


Figure 3: sparsidad de su red

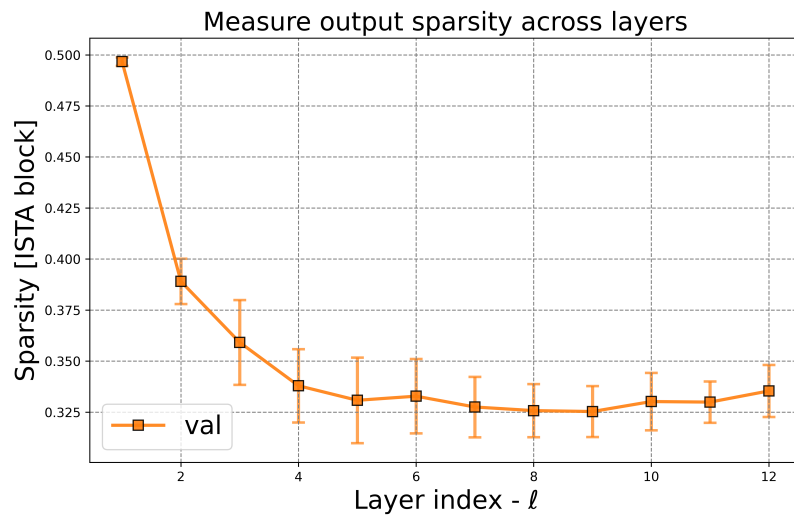


Figure 4: cr2 de mi red

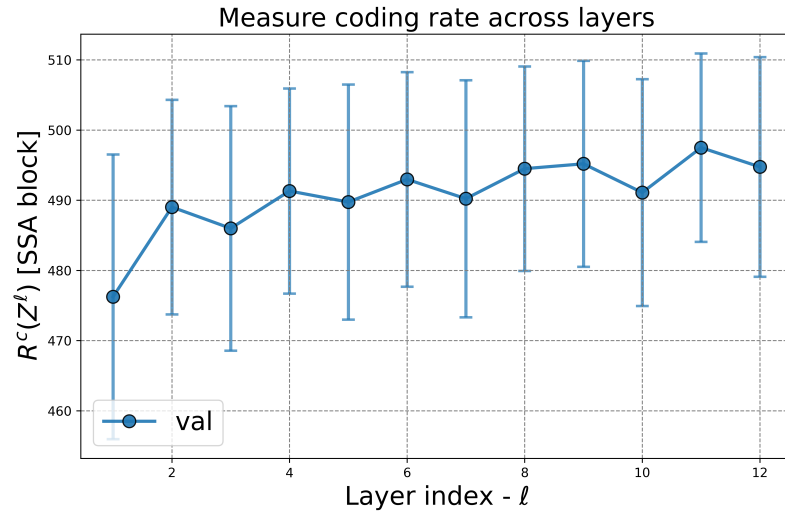


Figure 5: cr2 de su red

